

Section 82 Editorial — Wavelet Structures And Segment Tree Beats

A. Range K-th Value

How to find it

Stable-partition every level by one compressed-value bit and navigate with interval zero counts.

Correctness

At every level the mapped interval contains exactly the original query elements with the chosen prefix. Skipping the zero block when k exceeds its size preserves the rank of the desired element.

Complexity

$O((n+q) \log n)$ expected/amortized time and $O(n \log \sigma)$ or $O(n)$ structure memory, as appropriate.

C++

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
struct Wavelet {
    int B, n;
    vector<ll> vals;
    vector<vector<int>> z;
    vector<vector<ll>> zs;
    vector<int> mid;
    Wavelet(vector<ll> a) : n(a.size()) {
        vals = a;
        sort(vals.begin(), vals.end());
        vals.erase(unique(vals.begin(), vals.end()), vals.end());
        B = max(1, (int)bit_width((unsigned)max(1, (int)vals.size() - 1)));
        vector<int> r(n);
        for (int i = 0; i < n; i++)
            r[i] = lower_bound(vals.begin(), vals.end(), a[i]) - vals.begin();
        for (int b = B - 1; b >= 0; b--) {
            z.push_back(vector<int>(n + 1));
            zs.push_back(vector<ll>(n + 1));
            vector<int> r0, r1;
            vector<ll> a0, a1;
            for (int i = 0; i < n; i++) {
                bool zero = !(r[i] >> b & 1);
                z.back()[i + 1] = z.back()[i] + zero;
                zs.back()[i + 1] = zs.back()[i] + (zero ? a[i] : 0);
                (zero ? r0 : r1).push_back(r[i]);
                (zero ? a0 : a1).push_back(a[i]);
            }
            mid.push_back(r0.size());
            r0.insert(r0.end(), r1.begin(), r1.end());
            a0.insert(a0.end(), a1.begin(), a1.end());
        }
    }
};
```

```

        r.swap(r0);
        a.swap(a0);
    }
}

ll kth(int l, int r, int k) {
    int rank = 0;
    for (int d = 0; d < B; d++) {
        int q = z[d][r] - z[d][l], b = B - 1 - d;
        if (k <= q)
            l = z[d][l], r = z[d][r];
        else
            k -= q, rank |= 1 << b, l = mid[d] + l - z[d][l],
            r = mid[d] + r - z[d][r];
    }
    return vals[rank];
}

int lte(int l, int r, ll x) {
    int lim = upper_bound(vals.begin(), vals.end(), x) - vals.begin();
    if (lim == (int)vals.size())
        return r - 1;
    int ans = 0;
    for (int d = 0; d < B; d++) {
        int b = B - 1 - d;
        if (lim >> b & 1)
            ans += z[d][r] - z[d][l], l = mid[d] + l - z[d][l],
            r = mid[d] + r - z[d][r];
        else
            l = z[d][l], r = z[d][r];
    }
    return ans;
}

ll sumk(int l, int r, int k) {
    ll ans = 0;
    int rank = 0;
    for (int d = 0; d < B; d++) {
        int q = z[d][r] - z[d][l], b = B - 1 - d;
        if (k <= q)
            l = z[d][l], r = z[d][r];
        else
            ans += zs[d][r] - zs[d][l], k -= q, rank |= 1 << b,
            l = mid[d] + l - z[d][l], r = mid[d] + r - z[d][r];
    }
    return ans + vals[rank] * k;
}

};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<ll> a(n);
    for (auto &x : a)
        cin >> x;
    Wavelet w(a);
    while (q--) {
        int l, r, k;
        cin >> l >> r >> k;
        cout << w.kth(l - 1, r, k) << '\n';
    }
}

```

Python

```
import sys
from array import array

class Wavelet:
    def __init__(self, a):
        self.values = sorted(set(a))
        rank = {x: i for i, x in enumerate(self.values)}
        r = [rank[x] for x in a]
        self.bits = max(1, (len(self.values) - 1).bit_length())
        self.zero = []
        self.zero_sum = []
        self.mid = []
        v = a[:]
        for bit in range(self.bits - 1, -1, -1):
            pc = array("I", [0])
            ps = array("q", [0])
            zr = []
            zv = []
            orank = []
            oval = []
            for x, y in zip(r, v):
                z = not (x >> bit & 1)
                pc.append(pc[-1] + z)
                ps.append(ps[-1] + (y if z else 0))
                (zr if z else orank).append(x)
                (zv if z else oval).append(y)
            self.zero.append(pc)
            self.zero_sum.append(ps)
            self.mid.append(len(zr))
            r = zr + orank
            v = zv + oval

    def kth(self, l, r, k):
        rank = 0
        for level in range(self.bits):
            bit = self.bits - 1 - level
            p = self.zero[level]
            zl = p[r] - p[l]
            if k <= zl:
                l, r = p[l], p[r]
            else:
                k -= zl
                rank |= 1 << bit
                l, r = self.mid[level] + l - p[l], self.mid[level] + r - p[r]
        return self.values[rank]

    def lte(self, l, r, x):
        import bisect

        limit = bisect.bisect_right(self.values, x)
        if limit >= len(self.values):
            return r - l
        ans = 0
        for level in range(self.bits):
            bit = self.bits - 1 - level
            p = self.zero[level]
            if limit >> bit & 1:
                ans += p[r] - p[l]
                l, r = self.mid[level] + l - p[l], self.mid[level] + r - p[r]
            else:
```

```

        l, r = p[l], p[r]
    return ans

def sum_k(self, l, r, k):
    ans = 0
    rank = 0
    for level in range(self.bits):
        bit = self.bits - 1 - level
        p = self.zero[level]
        s = self.zero_sum[level]
        zl = p[r] - p[l]
        if k <= zl:
            l, r = p[l], p[r]
        else:
            ans += s[r] - s[l]
            k -= zl
            rank |= 1 << bit
            l, r = self.mid[level] + l - p[l], self.mid[level] + r - p[r]
    return ans + self.values[rank] * k

def main():
    d = list(map(int, sys.stdin.buffer.read().split()))
    n, q = d[:2]
    a = d[2 : 2 + n]
    w = Wavelet(a)
    at = 2 + n
    print(
        "\n".join(
            str(w.kth(d[i] - 1, d[i + 1], d[i + 2]))
            for i in range(at, at + 3 * q, 3)
        )
    )

if __name__ == "__main__":
    main()

```

B. Range Frequency Threshold

How to find it

Interpret the upper-bound rank bit by bit; add an entire zero branch whenever the bound takes the one branch.

Correctness

Every added branch has the first differing bit zero versus the bound's one and is therefore smaller; the continued branch contains all and only undecided values.

Complexity

$O((n+q) \log n)$ expected/amortized time and $O(n \log \sigma)$ or $O(n)$ structure memory, as appropriate.

C++

```

#include <bits/stdc++.h>
using namespace std;
using ll = long long;
struct Wavelet {
    int B, n;
    vector<ll> vals;
    vector<vector<int>> z;
    vector<vector<ll>> zs;
    vector<int> mid;
    Wavelet(vector<ll> a) : n(a.size()) {
        vals = a;
        sort(vals.begin(), vals.end());
        vals.erase(unique(vals.begin(), vals.end()), vals.end());
        B = max(1, (int)bit_width((unsigned)max(1, (int)vals.size() - 1)));
        vector<int> r(n);
        for (int i = 0; i < n; i++)
            r[i] = lower_bound(vals.begin(), vals.end(), a[i]) - vals.begin();
        for (int b = B - 1; b >= 0; b--) {
            z.push_back(vector<int>(n + 1));
            zs.push_back(vector<ll>(n + 1));
            vector<int> r0, r1;
            vector<ll> a0, a1;
            for (int i = 0; i < n; i++) {
                bool zero = !(r[i] >> b & 1);
                z.back()[i + 1] = z.back()[i] + zero;
                zs.back()[i + 1] = zs.back()[i] + (zero ? a[i] : 0);
                (zero ? r0 : r1).push_back(r[i]);
                (zero ? a0 : a1).push_back(a[i]);
            }
            mid.push_back(r0.size());
            r0.insert(r0.end(), r1.begin(), r1.end());
            a0.insert(a0.end(), a1.begin(), a1.end());
            r.swap(r0);
            a.swap(a0);
        }
    }
    ll kth(int l, int r, int k) {
        int rank = 0;
        for (int d = 0; d < B; d++) {
            int q = z[d][r] - z[d][l], b = B - 1 - d;
            if (k <= q)
                l = z[d][l], r = z[d][r];
            else
                k -= q, rank |= 1 << b, l = mid[d] + l - z[d][l],
                r = mid[d] + r - z[d][r];
        }
        return vals[rank];
    }
    int lte(int l, int r, ll x) {
        int lim = upper_bound(vals.begin(), vals.end(), x) - vals.begin();
        if (lim == (int)vals.size())
            return r - 1;
        int ans = 0;
        for (int d = 0; d < B; d++) {
            int b = B - 1 - d;
            if (lim >> b & 1)
                ans += z[d][r] - z[d][l], l = mid[d] + l - z[d][l],
                r = mid[d] + r - z[d][r];
            else
                l = z[d][l], r = z[d][r];
        }
        return ans;
    }
}

```

```

ll sumk(int l, int r, int k) {
    ll ans = 0;
    int rank = 0;
    for (int d = 0; d < B; d++) {
        int q = z[d][r] - z[d][l], b = B - 1 - d;
        if (k <= q)
            l = z[d][l], r = z[d][r];
        else
            ans += zs[d][r] - zs[d][l], k -= q, rank |= 1 << b,
            l = mid[d] + l - z[d][l], r = mid[d] + r - z[d][r];
    }
    return ans + vals[rank] * k;
}
};
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<ll> a(n);
    for (auto &x : a)
        cin >> x;
    Wavelet w(a);
    while (q--) {
        int l, r;
        ll x;
        cin >> l >> r >> x;
        cout << w.lte(l - 1, r, x) << '\n';
    }
}

```

Python

```

import sys
from array import array

class Wavelet:
    def __init__(self, a):
        self.values = sorted(set(a))
        rank = {x: i for i, x in enumerate(self.values)}
        r = [rank[x] for x in a]
        self.bits = max(1, (len(self.values) - 1).bit_length())
        self.zero = []
        self.zero_sum = []
        self.mid = []
        v = a[:]
        for bit in range(self.bits - 1, -1, -1):
            pc = array("I", [0])
            ps = array("q", [0])
            zr = []
            zv = []
            orank = []
            oval = []
            for x, y in zip(r, v):
                z = not (x >> bit & 1)
                pc.append(pc[-1] + z)
                ps.append(ps[-1] + (y if z else 0))
                (zr if z else orank).append(x)
                (zv if z else oval).append(y)
            self.zero.append(pc)

```

```

        self.zero_sum.append(ps)
        self.mid.append(len(zr))
        r = zr + orank
        v = zv + oval

def kth(self, l, r, k):
    rank = 0
    for level in range(self.bits):
        bit = self.bits - 1 - level
        p = self.zero[level]
        zl = p[r] - p[l]
        if k <= zl:
            l, r = p[l], p[r]
        else:
            k -= zl
            rank |= 1 << bit
            l, r = self.mid[level] + l - p[l], self.mid[level] + r - p[r]
    return self.values[rank]

def lte(self, l, r, x):
    import bisect

    limit = bisect.bisect_right(self.values, x)
    if limit >= len(self.values):
        return r - l
    ans = 0
    for level in range(self.bits):
        bit = self.bits - 1 - level
        p = self.zero[level]
        if limit >> bit & 1:
            ans += p[r] - p[l]
            l, r = self.mid[level] + l - p[l], self.mid[level] + r - p[r]
        else:
            l, r = p[l], p[r]
    return ans

def sum_k(self, l, r, k):
    ans = 0
    rank = 0
    for level in range(self.bits):
        bit = self.bits - 1 - level
        p = self.zero[level]
        s = self.zero_sum[level]
        zl = p[r] - p[l]
        if k <= zl:
            l, r = p[l], p[r]
        else:
            ans += s[r] - s[l]
            k -= zl
            rank |= 1 << bit
            l, r = self.mid[level] + l - p[l], self.mid[level] + r - p[r]
    return ans + self.values[rank] * k

def main():
    d = list(map(int, sys.stdin.buffer.read().split()))
    n, q = d[:2]
    a = d[2 : 2 + n]
    w = Wavelet(a)
    at = 2 + n
    print(
        "\n".join(
            str(w.lte(d[i] - 1, d[i + 1], d[i + 2]))

```

```

        for i in range(at, at + 3 * q, 3)
    )
)

if __name__ == "__main__":
    main()

```

C. Range Cap And Sum

How to find it

Store the largest value, second largest value, its multiplicity, and the sum.

Correctness

When the cap lies above the second maximum, exactly the maximum occurrences change, so the node update is exact. Otherwise recursion partitions the affected value classes until that condition holds.

Complexity

$O((n+q) \log n)$ expected/amortized time and $O(n \log \sigma)$ or $O(n)$ structure memory, as appropriate.

C++

```

#include <bits/stdc++.h>
using namespace std;
using ll = long long;
const ll INF = 4e18;
struct Beats {
    struct N {
        ll sum = 0, mx = -INF, smx = -INF, mn = INF, smn = INF;
        int mxc = 0, mnc = 0;
    };
    int n;
    vector<N> t;
    Beats(vector<ll> a) : n(a.size()), t(4 * n) { build(1, 0, n, a); }
    void build(int p, int l, int r, vector<ll> &a) {
        if (r - l == 1) {
            t[p] = {a[l], a[l], -INF, a[l], INF, 1, 1};
            return;
        }
        int m = (l + r) / 2;
        build(p * 2, l, m, a);
        build(p * 2 + 1, m, r, a);
        pull(p);
    }
    void pull(int p) {
        N &a = t[p * 2], &b = t[p * 2 + 1], &x = t[p];
        x.sum = a.sum + b.sum;
        if (a.mx > b.mx)
            x.mx = a.mx, x.mxc = a.mxc, x.smx = max(a.smx, b.mx);
        else if (a.mx < b.mx)
            x.mx = b.mx, x.mxc = b.mxc, x.smx = max(a.mx, b.smx);
        else
            x.mx = a.mx, x.mxc = a.mxc + b.mxc, x.smx = max(a.smx, b.smx);
        if (a.mn < b.mn)
            x.mn = a.mn, x.mnc = a.mnc, x.smn = min(a.smn, b.mn);
    }
};

```

```

    else if (a.mn > b.mn)
        x.mn = b.mn, x.mnc = b.mnc, x.smn = min(a.mn, b.smn);
    else
        x.mn = a.mn, x.mnc = a.mnc + b.mnc, x.smn = min(a.smn, b.smn);
}

void upper_node(int p, ll x) {
    if (t[p].mx <= x)
        return;
    t[p].sum += (x - t[p].mx) * t[p].mxc;
    if (t[p].mn == t[p].mx)
        t[p].mn = x;
    else if (t[p].smn == t[p].mx)
        t[p].smn = x;
    t[p].mx = x;
}

void lower_node(int p, ll x) {
    if (t[p].mn >= x)
        return;
    t[p].sum += (x - t[p].mn) * t[p].mnc;
    if (t[p].mx == t[p].mn)
        t[p].mx = x;
    else if (t[p].smx == t[p].mn)
        t[p].smx = x;
    t[p].mn = x;
}

void push(int p) {
    upper_node(p * 2, t[p].mx);
    upper_node(p * 2 + 1, t[p].mx);
    lower_node(p * 2, t[p].mn);
    lower_node(p * 2 + 1, t[p].mn);
}

void upper(int ql, int qr, ll x, int p = 1, int l = 0, int r = -1) {
    if (r < 0)
        r = n;
    if (qr <= l || r <= ql || t[p].mx <= x)
        return;
    if (ql <= l && r <= qr && t[p].smx < x) {
        upper_node(p, x);
        return;
    }
    push(p);
    int m = (l + r) / 2;
    upper(ql, qr, x, p * 2, l, m);
    upper(ql, qr, x, p * 2 + 1, m, r);
    pull(p);
}

void lower(int ql, int qr, ll x, int p = 1, int l = 0, int r = -1) {
    if (r < 0)
        r = n;
    if (qr <= l || r <= ql || t[p].mn >= x)
        return;
    if (ql <= l && r <= qr && t[p].smn > x) {
        lower_node(p, x);
        return;
    }
    push(p);
    int m = (l + r) / 2;
    lower(ql, qr, x, p * 2, l, m);
    lower(ql, qr, x, p * 2 + 1, m, r);
    pull(p);
}

ll query(int ql, int qr, int p = 1, int l = 0, int r = -1) {
    if (r < 0)

```

```

    r = n;
    if (qr <= 1 || r <= ql)
        return 0;
    if (ql <= 1 && r <= qr)
        return t[p].sum;
    push(p);
    int m = (l + r) / 2;
    return query(ql, qr, p * 2, l, m) + query(ql, qr, p * 2 + 1, m, r);
}
};

struct ModTree {
    int n;
    vector<ll> s, mx;
    ModTree(vector<ll> a) : n(a.size()), s(4 * n), mx(4 * n) {
        build(1, 0, n, a);
    }
    void build(int p, int l, int r, vector<ll> &a) {
        if (r - l == 1) {
            s[p] = mx[p] = a[l];
            return;
        }
        int m = (l + r) / 2;
        build(p * 2, l, m, a);
        build(p * 2 + 1, m, r, a);
        pull(p);
    }
    void pull(int p) {
        s[p] = s[p * 2] + s[p * 2 + 1];
        mx[p] = max(mx[p * 2], mx[p * 2 + 1]);
    }
    void mod(int ql, int qr, ll x, int p = 1, int l = 0, int r = -1) {
        if (r < 0)
            r = n;
        if (qr <= 1 || r <= ql || mx[p] < x)
            return;
        if (r - l == 1) {
            s[p] %= x;
            mx[p] = s[p];
            return;
        }
        int m = (l + r) / 2;
        mod(ql, qr, x, p * 2, l, m);
        mod(ql, qr, x, p * 2 + 1, m, r);
        pull(p);
    }
    void setv(int i, ll x, int p = 1, int l = 0, int r = -1) {
        if (r < 0)
            r = n;
        if (r - l == 1) {
            s[p] = mx[p] = x;
            return;
        }
        int m = (l + r) / 2;
        i < m ? setv(i, x, p * 2, l, m) : setv(i, x, p * 2 + 1, m, r);
        pull(p);
    }
    ll query(int ql, int qr, int p = 1, int l = 0, int r = -1) {
        if (r < 0)
            r = n;
        if (qr <= 1 || r <= ql)
            return 0;
        if (ql <= 1 && r <= qr)
            return s[p];
    }
};

```

```

    int m = (l + r) / 2;
    return query(ql, qr, p * 2, l, m) + query(ql, qr, p * 2 + 1, m, r);
}
};
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<ll> a(n);
    for (auto &x : a)
        cin >> x;
    Beats t(a);
    while (q--) {
        string op;
        int l, r;
        cin >> op >> l >> r;
        if (op == "CAP") {
            ll x;
            cin >> x;
            t.upper(l - 1, r, x);
        } else
            cout << t.query(l - 1, r) << '\n';
    }
}

```

Python

```

import sys

INF = 10**30

class Beats:
    def __init__(self, a):
        n = len(a)
        z = 4 * n
        self.n = n
        self.sum = [0] * z
        self.mx = [-INF] * z
        self.smx = [-INF] * z
        self.mxc = [0] * z
        self.mn = [INF] * z
        self.smn = [INF] * z
        self.mnc = [0] * z
        self._build(1, 0, n, a)

    def _build(self, p, l, r, a):
        if r - l == 1:
            self.sum[p] = self.mx[p] = self.mn[p] = a[l]
            self.mxc[p] = self.mnc[p] = 1
            return
        m = (l + r) // 2
        self._build(p * 2, l, m, a)
        self._build(p * 2 + 1, m, r, a)
        self._pull(p)

    def _pull(self, p):
        a, b = p * 2, p * 2 + 1
        self.sum[p] = self.sum[a] + self.sum[b]
        if self.mx[a] > self.mx[b]:

```

```

        self.mx[p], self.mxc[p], self.smx[p] = (
            self.mx[a],
            self.mxc[a],
            max(self.smx[a], self.mx[b]),
        )
    elif self.mx[a] < self.mx[b]:
        self.mx[p], self.mxc[p], self.smx[p] = (
            self.mx[b],
            self.mxc[b],
            max(self.mx[a], self.smx[b]),
        )
    else:
        self.mx[p], self.mxc[p], self.smx[p] = (
            self.mx[a],
            self.mxc[a] + self.mxc[b],
            max(self.smx[a], self.smx[b]),
        )
    if self.mn[a] < self.mn[b]:
        self.mn[p], self.mnc[p], self.smn[p] = (
            self.mn[a],
            self.mnc[a],
            min(self.smn[a], self.mn[b]),
        )
    elif self.mn[a] > self.mn[b]:
        self.mn[p], self.mnc[p], self.smn[p] = (
            self.mn[b],
            self.mnc[b],
            min(self.mn[a], self.smn[b]),
        )
    else:
        self.mn[p], self.mnc[p], self.smn[p] = (
            self.mn[a],
            self.mnc[a] + self.mnc[b],
            min(self.smn[a], self.smn[b]),
        )

def _upper_node(self, p, x):
    if self.mx[p] <= x:
        return
    self.sum[p] += (x - self.mx[p]) * self.mxc[p]
    if self.mn[p] == self.mx[p]:
        self.mn[p] = x
    elif self.smn[p] == self.mx[p]:
        self.smn[p] = x
    self.mx[p] = x

def _lower_node(self, p, x):
    if self.mn[p] >= x:
        return
    self.sum[p] += (x - self.mn[p]) * self.mnc[p]
    if self.mx[p] == self.mn[p]:
        self.mx[p] = x
    elif self.smx[p] == self.mn[p]:
        self.smx[p] = x
    self.mn[p] = x

def _push(self, p):
    for c in (p * 2, p * 2 + 1):
        self._upper_node(c, self.mx[p])
        self._lower_node(c, self.mn[p])

def upper(self, ql, qr, x, p=1, l=0, r=None):
    if r is None:

```

```

        r = self.n
    if qr <= l or r <= ql or self.mx[p] <= x:
        return
    if ql <= l and r <= qr and self.smx[p] < x:
        self._upper_node(p, x)
        return
    self._push(p)
    m = (l + r) // 2
    self.upper(ql, qr, x, p * 2, l, m)
    self.upper(ql, qr, x, p * 2 + 1, m, r)
    self._pull(p)

def lower(self, ql, qr, x, p=1, l=0, r=None):
    if r is None:
        r = self.n
    if qr <= l or r <= ql or self.mn[p] >= x:
        return
    if ql <= l and r <= qr and self.smn[p] > x:
        self._lower_node(p, x)
        return
    self._push(p)
    m = (l + r) // 2
    self.lower(ql, qr, x, p * 2, l, m)
    self.lower(ql, qr, x, p * 2 + 1, m, r)
    self._pull(p)

def query(self, ql, qr, p=1, l=0, r=None):
    if r is None:
        r = self.n
    if qr <= l or r <= ql:
        return 0
    if ql <= l and r <= qr:
        return self.sum[p]
    self._push(p)
    m = (l + r) // 2
    return self.query(ql, qr, p * 2, l, m) + self.query(
        ql, qr, p * 2 + 1, m, r
    )

```

```

class ModTree:
    def __init__(self, a):
        self.n = len(a)
        self.s = [0] * (4 * self.n)
        self.mx = [0] * (4 * self.n)
        self._build(1, 0, self.n, a)

    def _build(self, p, l, r, a):
        if r - l == 1:
            self.s[p] = self.mx[p] = a[l]
            return
        m = (l + r) // 2
        self._build(p * 2, l, m, a)
        self._build(p * 2 + 1, m, r, a)
        self._pull(p)

    def _pull(self, p):
        self.s[p] = self.s[p * 2] + self.s[p * 2 + 1]
        self.mx[p] = max(self.mx[p * 2], self.mx[p * 2 + 1])

    def mod(self, ql, qr, x, p=1, l=0, r=None):
        if r is None:
            r = self.n

```

```

    if qr <= l or r <= ql or self.mx[p] < x:
        return
    if r - l == 1:
        self.s[p] %= x
        self.mx[p] = self.s[p]
        return
    m = (l + r) // 2
    self.mod(ql, qr, x, p * 2, l, m)
    self.mod(ql, qr, x, p * 2 + 1, m, r)
    self._pull(p)

def set(self, i, x, p=1, l=0, r=None):
    if r is None:
        r = self.n
    if r - l == 1:
        self.s[p] = self.mx[p] = x
        return
    m = (l + r) // 2
    if i < m:
        self.set(i, x, p * 2, l, m)
    else:
        self.set(i, x, p * 2 + 1, m, r)
    self._pull(p)

def query(self, ql, qr, p=1, l=0, r=None):
    if r is None:
        r = self.n
    if qr <= l or r <= ql:
        return 0
    if ql <= l and r <= qr:
        return self.s[p]
    m = (l + r) // 2
    return self.query(ql, qr, p * 2, l, m) + self.query(
        ql, qr, p * 2 + 1, m, r
    )

def main():
    d = sys.stdin.buffer.read().split()
    n, q = map(int, d[:2])
    t = Beats(list(map(int, d[2 : 2 + n])))
    at = 2 + n
    out = []
    for _ in range(q):
        op = d[at]
        l = int(d[at + 1]) - 1
        r = int(d[at + 2])
        at += 3
        if op == b"CAP":
            t.upper(l, r, int(d[at]))
            at += 1
        else:
            out.append(str(t.query(l, r)))
    print("\n".join(out))

if __name__ == "__main__":
    main()

```

D. Range Modulo And Sum

How to find it

A modulo update does nothing to a segment whose maximum is below the modulus.

Correctness

Pruned nodes contain no changed element. At reached leaves the exact modulo is applied, and pulling reconstructs exact sums and maxima.

Complexity

$O((n+q) \log n)$ expected/amortized time and $O(n \log \sigma)$ or $O(n)$ structure memory, as appropriate.

C++

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
const ll INF = 4e18;
struct Beats {
    struct N {
        ll sum = 0, mx = -INF, smx = -INF, mn = INF, smn = INF;
        int mxc = 0, mnc = 0;
    };
    int n;
    vector<N> t;
    Beats(vector<ll> a) : n(a.size()), t(4 * n) { build(1, 0, n, a); }
    void build(int p, int l, int r, vector<ll> &a) {
        if (r - l == 1) {
            t[p] = {a[l], a[l], -INF, a[l], INF, 1, 1};
            return;
        }
        int m = (l + r) / 2;
        build(p * 2, l, m, a);
        build(p * 2 + 1, m, r, a);
        pull(p);
    }
    void pull(int p) {
        N &a = t[p * 2], &b = t[p * 2 + 1], &x = t[p];
        x.sum = a.sum + b.sum;
        if (a.mx > b.mx)
            x.mx = a.mx, x.mxc = a.mxc, x.smx = max(a.smx, b.mx);
        else if (a.mx < b.mx)
            x.mx = b.mx, x.mxc = b.mxc, x.smx = max(a.mx, b.smx);
        else
            x.mx = a.mx, x.mxc = a.mxc + b.mxc, x.smx = max(a.smx, b.smx);
        if (a.mn < b.mn)
            x.mn = a.mn, x.mnc = a.mnc, x.smn = min(a.smn, b.mn);
        else if (a.mn > b.mn)
            x.mn = b.mn, x.mnc = b.mnc, x.smn = min(a.mn, b.smn);
        else
            x.mn = a.mn, x.mnc = a.mnc + b.mnc, x.smn = min(a.smn, b.smn);
    }
    void upper_node(int p, ll x) {
        if (t[p].mx <= x)
            return;
        t[p].sum += (x - t[p].mx) * t[p].mxc;
        if (t[p].mn == t[p].mx)
```

```

    t[p].mn = x;
else if (t[p].smn == t[p].mx)
    t[p].smn = x;
t[p].mx = x;
}

void lower_node(int p, ll x) {
    if (t[p].mn >= x)
        return;
    t[p].sum += (x - t[p].mn) * t[p].mnc;
    if (t[p].mx == t[p].mn)
        t[p].mx = x;
    else if (t[p].smx == t[p].mn)
        t[p].smx = x;
    t[p].mn = x;
}

void push(int p) {
    upper_node(p * 2, t[p].mx);
    upper_node(p * 2 + 1, t[p].mx);
    lower_node(p * 2, t[p].mn);
    lower_node(p * 2 + 1, t[p].mn);
}

void upper(int ql, int qr, ll x, int p = 1, int l = 0, int r = -1) {
    if (r < 0)
        r = n;
    if (qr <= l || r <= ql || t[p].mx <= x)
        return;
    if (ql <= l && r <= qr && t[p].smx < x) {
        upper_node(p, x);
        return;
    }
    push(p);
    int m = (l + r) / 2;
    upper(ql, qr, x, p * 2, l, m);
    upper(ql, qr, x, p * 2 + 1, m, r);
    pull(p);
}

void lower(int ql, int qr, ll x, int p = 1, int l = 0, int r = -1) {
    if (r < 0)
        r = n;
    if (qr <= l || r <= ql || t[p].mn >= x)
        return;
    if (ql <= l && r <= qr && t[p].smn > x) {
        lower_node(p, x);
        return;
    }
    push(p);
    int m = (l + r) / 2;
    lower(ql, qr, x, p * 2, l, m);
    lower(ql, qr, x, p * 2 + 1, m, r);
    pull(p);
}

ll query(int ql, int qr, int p = 1, int l = 0, int r = -1) {
    if (r < 0)
        r = n;
    if (qr <= l || r <= ql)
        return 0;
    if (ql <= l && r <= qr)
        return t[p].sum;
    push(p);
    int m = (l + r) / 2;
    return query(ql, qr, p * 2, l, m) + query(ql, qr, p * 2 + 1, m, r);
}
};

```

```

struct ModTree {
    int n;
    vector<ll> s, mx;
    ModTree(vector<ll> a) : n(a.size()), s(4 * n), mx(4 * n) {
        build(1, 0, n, a);
    }
    void build(int p, int l, int r, vector<ll> &a) {
        if (r - l == 1) {
            s[p] = mx[p] = a[l];
            return;
        }
        int m = (l + r) / 2;
        build(p * 2, l, m, a);
        build(p * 2 + 1, m, r, a);
        pull(p);
    }
    void pull(int p) {
        s[p] = s[p * 2] + s[p * 2 + 1];
        mx[p] = max(mx[p * 2], mx[p * 2 + 1]);
    }
    void mod(int ql, int qr, ll x, int p = 1, int l = 0, int r = -1) {
        if (r < 0)
            r = n;
        if (qr <= l || r <= ql || mx[p] < x)
            return;
        if (r - l == 1) {
            s[p] %= x;
            mx[p] = s[p];
            return;
        }
        int m = (l + r) / 2;
        mod(ql, qr, x, p * 2, l, m);
        mod(ql, qr, x, p * 2 + 1, m, r);
        pull(p);
    }
    void setv(int i, ll x, int p = 1, int l = 0, int r = -1) {
        if (r < 0)
            r = n;
        if (r - l == 1) {
            s[p] = mx[p] = x;
            return;
        }
        int m = (l + r) / 2;
        i < m ? setv(i, x, p * 2, l, m) : setv(i, x, p * 2 + 1, m, r);
        pull(p);
    }
    ll query(int ql, int qr, int p = 1, int l = 0, int r = -1) {
        if (r < 0)
            r = n;
        if (qr <= l || r <= ql)
            return 0;
        if (ql <= l && r <= qr)
            return s[p];
        int m = (l + r) / 2;
        return query(ql, qr, p * 2, l, m) + query(ql, qr, p * 2 + 1, m, r);
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<ll> a(n);

```

```

for (auto &x : a)
    cin >> x;
ModTree t(a);
while (q--) {
    string op;
    cin >> op;
    if (op == "SET") {
        int i;
        ll x;
        cin >> i >> x;
        t.setv(i - 1, x);
    } else {
        int l, r;
        cin >> l >> r;
        if (op == "MOD") {
            ll x;
            cin >> x;
            t.mod(l - 1, r, x);
        } else
            cout << t.query(l - 1, r) << '\n';
    }
}
}
}

```

Python

```

import sys

INF = 10**30

class Beats:
    def __init__(self, a):
        n = len(a)
        z = 4 * n
        self.n = n
        self.sum = [0] * z
        self.mx = [-INF] * z
        self.smx = [-INF] * z
        self.mxc = [0] * z
        self.mn = [INF] * z
        self.smn = [INF] * z
        self.mnc = [0] * z
        self._build(1, 0, n, a)

    def _build(self, p, l, r, a):
        if r - l == 1:
            self.sum[p] = self.mx[p] = self.mn[p] = a[l]
            self.mxc[p] = self.mnc[p] = 1
            return
        m = (l + r) // 2
        self._build(p * 2, l, m, a)
        self._build(p * 2 + 1, m, r, a)
        self._pull(p)

    def _pull(self, p):
        a, b = p * 2, p * 2 + 1
        self.sum[p] = self.sum[a] + self.sum[b]
        if self.mx[a] > self.mx[b]:
            self.mx[p], self.mxc[p], self.smx[p] = (
                self.mx[a],

```

```

        self.mxc[a],
        max(self.smx[a], self.mx[b]),
    )
elif self.mx[a] < self.mx[b]:
    self.mx[p], self.mxc[p], self.smx[p] = (
        self.mx[b],
        self.mxc[b],
        max(self.mx[a], self.smx[b]),
    )
else:
    self.mx[p], self.mxc[p], self.smx[p] = (
        self.mx[a],
        self.mxc[a] + self.mxc[b],
        max(self.smx[a], self.smx[b]),
    )
if self.mn[a] < self.mn[b]:
    self.mn[p], self.mnc[p], self.smn[p] = (
        self.mn[a],
        self.mnc[a],
        min(self.smn[a], self.mn[b]),
    )
elif self.mn[a] > self.mn[b]:
    self.mn[p], self.mnc[p], self.smn[p] = (
        self.mn[b],
        self.mnc[b],
        min(self.mn[a], self.smn[b]),
    )
else:
    self.mn[p], self.mnc[p], self.smn[p] = (
        self.mn[a],
        self.mnc[a] + self.mnc[b],
        min(self.smn[a], self.smn[b]),
    )

def _upper_node(self, p, x):
    if self.mx[p] <= x:
        return
    self.sum[p] += (x - self.mx[p]) * self.mxc[p]
    if self.mn[p] == self.mx[p]:
        self.mn[p] = x
    elif self.smn[p] == self.mx[p]:
        self.smn[p] = x
    self.mx[p] = x

def _lower_node(self, p, x):
    if self.mn[p] >= x:
        return
    self.sum[p] += (x - self.mn[p]) * self.mnc[p]
    if self.mx[p] == self.mn[p]:
        self.mx[p] = x
    elif self.smx[p] == self.mn[p]:
        self.smx[p] = x
    self.mn[p] = x

def _push(self, p):
    for c in (p * 2, p * 2 + 1):
        self._upper_node(c, self.mx[p])
        self._lower_node(c, self.mn[p])

def upper(self, ql, qr, x, p=1, l=0, r=None):
    if r is None:
        r = self.n
    if qr <= l or r <= ql or self.mx[p] <= x:

```

```

        return
    if ql <= l and r <= qr and self.smx[p] < x:
        self._upper_node(p, x)
        return
    self._push(p)
    m = (l + r) // 2
    self.upper(ql, qr, x, p * 2, l, m)
    self.upper(ql, qr, x, p * 2 + 1, m, r)
    self._pull(p)

def lower(self, ql, qr, x, p=1, l=0, r=None):
    if r is None:
        r = self.n
    if qr <= l or r <= ql or self.mn[p] >= x:
        return
    if ql <= l and r <= qr and self.smn[p] > x:
        self._lower_node(p, x)
        return
    self._push(p)
    m = (l + r) // 2
    self.lower(ql, qr, x, p * 2, l, m)
    self.lower(ql, qr, x, p * 2 + 1, m, r)
    self._pull(p)

def query(self, ql, qr, p=1, l=0, r=None):
    if r is None:
        r = self.n
    if qr <= l or r <= ql:
        return 0
    if ql <= l and r <= qr:
        return self.sum[p]
    self._push(p)
    m = (l + r) // 2
    return self.query(ql, qr, p * 2, l, m) + self.query(
        ql, qr, p * 2 + 1, m, r
    )

class ModTree:
    def __init__(self, a):
        self.n = len(a)
        self.s = [0] * (4 * self.n)
        self.mx = [0] * (4 * self.n)
        self._build(1, 0, self.n, a)

    def _build(self, p, l, r, a):
        if r - l == 1:
            self.s[p] = self.mx[p] = a[l]
            return
        m = (l + r) // 2
        self._build(p * 2, l, m, a)
        self._build(p * 2 + 1, m, r, a)
        self._pull(p)

    def _pull(self, p):
        self.s[p] = self.s[p * 2] + self.s[p * 2 + 1]
        self.mx[p] = max(self.mx[p * 2], self.mx[p * 2 + 1])

    def mod(self, ql, qr, x, p=1, l=0, r=None):
        if r is None:
            r = self.n
        if qr <= l or r <= ql or self.mx[p] < x:
            return

```

```

        if r - l == 1:
            self.s[p] %= x
            self.mx[p] = self.s[p]
            return
        m = (l + r) // 2
        self.mod(ql, qr, x, p * 2, l, m)
        self.mod(ql, qr, x, p * 2 + 1, m, r)
        self._pull(p)

def set(self, i, x, p=1, l=0, r=None):
    if r is None:
        r = self.n
    if r - l == 1:
        self.s[p] = self.mx[p] = x
        return
    m = (l + r) // 2
    if i < m:
        self.set(i, x, p * 2, l, m)
    else:
        self.set(i, x, p * 2 + 1, m, r)
    self._pull(p)

def query(self, ql, qr, p=1, l=0, r=None):
    if r is None:
        r = self.n
    if qr <= l or r <= ql:
        return 0
    if ql <= l and r <= qr:
        return self.s[p]
    m = (l + r) // 2
    return self.query(ql, qr, p * 2, l, m) + self.query(
        ql, qr, p * 2 + 1, m, r
    )

def main():
    d = sys.stdin.buffer.read().split()
    n, q = map(int, d[:2])
    t = ModTree(list(map(int, d[2 : 2 + n])))
    at = 2 + n
    out = []
    for _ in range(q):
        op = d[at]
        at += 1
        if op == b"SET":
            t.set(int(d[at]) - 1, int(d[at + 1]))
            at += 2
        else:
            l = int(d[at]) - 1
            r = int(d[at + 1])
            at += 2
            if op == b"MOD":
                t.mod(l, r, int(d[at]))
                at += 1
            else:
                out.append(str(t.query(l, r)))
    print("\n".join(out))

if __name__ == "__main__":
    main()

```

Upsolving and stress testing

For A/B, sort each literal query slice and answer rank/frequency directly; include duplicates and thresholds absent from compression. For C/D, mutate every element in the requested range and recompute sums. Range modulo uses the pruning fact `maximum < modulus`; a sum or average cannot certify that no element changes. Segment-tree beats similarly relies on exact extremum-class state, not optimistic average behavior.

Carry forward the boundary distinction: wavelet levels navigate a static value domain, while beats stores just enough extremal information to prove when a nonlinear update is uniform on a node.